Planning a Software Project

Learning Outcomes

By the end of this lesson, students should be able to:

- Explain why project planning is critical
- Define a software problem clearly
- Identify project goals and requirements
- Develop a solution strategy
- Connect planning to software engineering principles

What is Project Planning?

Project planning is:

1. **Understanding What Needs to Be Built:** This is the foundation of planning. It involves deep analysis of the problem domain, stakeholder needs, and desired outcomes.

2. Identifying Constraints and Scope: Constraints are limitations you must work within; scope defines boundaries of what is and is not included.

3. Clarifying Expectations: Ensures all stakeholders (clients, users, developers, management) have aligned understanding of goals, deliverables, timelines, and responsibilities.

4. Reducing Uncertainty Before Development: Software development is inherently uncertain; planning systematically identifies and mitigates risks.

Defining the Problem

Defining the problem means:

- **Clearly stating the issue to be solved:** This means moving beyond vague complaints ("The system is too slow") to a precise, unambiguous problem statement that can guide the entire project.

Without a clear problem statement, solutions may address symptoms instead of root causes, or teams may solve different problems altogether.

- **Identifying users or stakeholders:** Determining who is affected by the problem and who will be affected by the solution.

Different stakeholders have different needs and success criteria. Missing a key stakeholder group leads to solutions that fail in practice.

Defining the Problem

- **Understanding the environment:** Examining the context in which the problem exists and the solution will operate.

Key aspects:

Technical environment: Existing systems, platforms, infrastructure

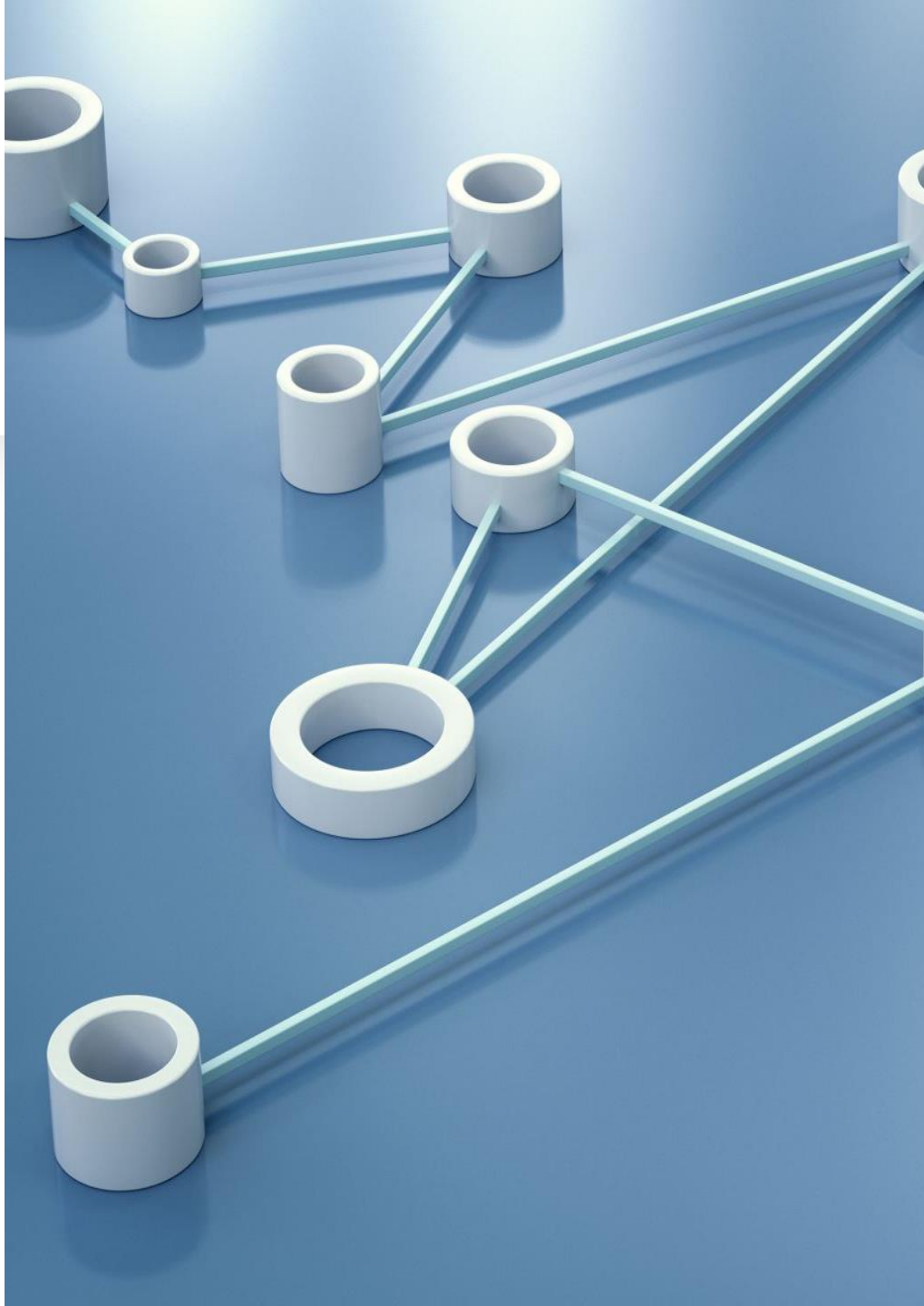
Organizational environment: Culture, processes, politics

Physical environment: Hardware constraints, location factors

Market/competitive environment: Industry trends, competitor solutions

A perfect technical solution can fail if it doesn't fit the environment.

Example: A cloud-based solution won't work in environments with strict data sovereignty laws.



Defining the Problem

- **Determining current limitations:** Identifying the boundaries and constraints that currently exist and may limit potential solutions.

Types of limitations:

Technical: Legacy systems, compatibility requirements

Resource: Budget, time, skill availability

Regulatory: Compliance requirements, legal restrictions

Cultural: Resistance to change

Understanding limitations prevents proposing unrealistic solutions and helps identify which constraints can be challenged versus which must be accepted.

Why Problem Definition Matters

Without clear problem definition:

- **Wrong software may be built:** Solving a problem that doesn't exist, solving the wrong problem, or building a solution that doesn't address the real need.

- **Requirements become unstable:** Problem definition is the foundation for all downstream requirements. Without it, requirements lack an anchor.

  Unstable requirements lead to rework, missed deadlines, and frustrated teams who feel they're building on shifting sand.

- **Time and money are wasted:** The later a problem is found, the more costly it is to fix. Projects with poor problem definition often run over budget, exceed timelines, or get cancelled after significant investment.

Why Problem Definition Matters

- **Teams misunderstand expectations:** Different interpretations of an unclear problem lead to misaligned efforts.

  Misalignment causes conflict, duplicated work, missed dependencies, and ultimately, a product that satisfies no one completely.

Poor problem definition doesn't just affect the initial phase — it creates compounding problems throughout the project lifecycle:

Problem Definition Techniques

- **Stakeholder interviews:** Direct conversations with individuals affected by the problem to gather diverse perspectives and uncover unspoken needs.
- **Surveys and questionnaires:** Structured tools to collect quantitative data and broad feedback from a larger group efficiently.
- **Observation of current systems** Watching real users interact with existing processes to identify friction points and workarounds firsthand.
- **Reviewing existing documentation:** Analyzing reports, logs, manuals, and prior project artifacts to understand historical context and formal constraints.
- **Brainstorming sessions:** Collaborative group activities that generate creative insights and surface assumptions by encouraging open idea sharing.

GOALS AND REQUIREMENTS

- **Goals** = What We Want to Achieve

Goals are high-level, strategic outcomes that describe the overall purpose or desired impact of the system.

Characteristics:

- Broad and visionary,

- Often business-oriented or user-focused

- Not directly testable

- Provide direction and motivation

Example:

"Improve patient satisfaction by reducing clinic wait times."

- **Requirements** = What the System Must Do

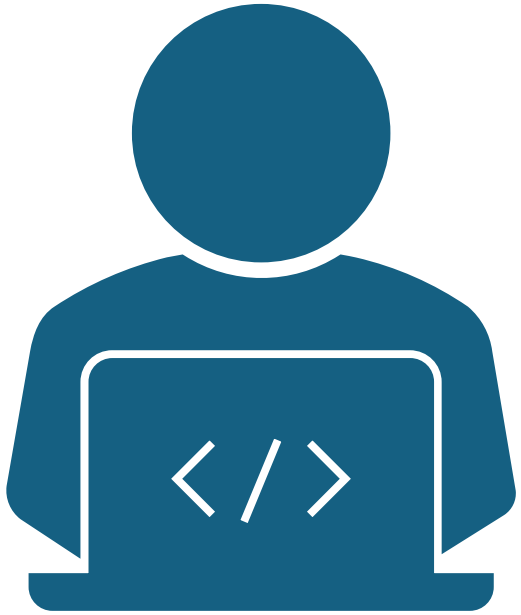
Requirements are specific, detailed conditions the system must satisfy to achieve the goals.

Characteristics:

- Precise and unambiguous
- Technically actionable
- Must be testable/verifiable
- Can be traced back to goals

Example:

"The system shall allow clinic staff to check in patients in under 60 seconds using a tablet interface."



Types of Requirements

Functional Requirements

What the system should do:

- Login system
- Generate reports
- Store data

Non-Functional Requirements

How the system should perform:

- Fast response time
- Secure data
- Easy to use

Characteristics of Good Requirements

Good requirements are:

- **Clear** – Written in simple, straightforward language that is easily understood by all stakeholders, both technical and non-technical.
- **Measurable** – Quantifiable with specific criteria for success, so completion can be evaluated objectively.
- **Testable** – Able to be verified through inspection, demonstration, analysis, or test to confirm the requirement is met.
- **Unambiguous** – Has only one possible interpretation to prevent different understandings among team members.
- **Feasible** – Realistically achievable within known project constraints like time, budget, technology, and resources.

Sources of Requirements Gathering

- **Users** – The end-users of the software who provide direct feedback on functionality, usability, and daily workflow needs.
- **Clients** – The individuals or organizations funding the project who define business objectives, constraints, and high-level expectations.
- **Domain experts** – Subject matter specialists who contribute deep knowledge about the industry, processes, and technical intricacies of the problem area.
- **Existing systems** – Legacy applications or competitor products that reveal implicit needs, gaps to fill, and integration opportunities.
- **Regulations and policies** – Legal, compliance, or organizational rules that impose mandatory constraints and standards on the solution.

SOLUTION STRATEGY

What is a Solution Strategy?

- Solution Strategy = Plan for Solving the Problem
- A high-level blueprint that outlines how we will approach building the solution—the bridge between defining the problem (what and why) and executing the project (how).

- This includes;

- Technology Choices: Selecting the specific software tools, languages, frameworks, and platforms that best fit the problem, team expertise, and long-term maintainability.
E.g; Choosing between React vs. Angular, cloud providers, database types, or third-party APIs.

What is a Solution Strategy?

- Architecture Approach: Defining the overall structure of the system: how components will interact, major layers, communication patterns, and key design principles.

Example: Monolithic vs. microservices, client-server vs. peer-to-peer, event-driven architecture.

Architecture determines scalability, resilience, and how easily the system can evolve.

- Development Model: Choosing the software development lifecycle methodology that guides the project's flow, team collaboration, and progress tracking.

Example: Waterfall, Agile, Hybrid, or iterative models.

This shapes how work is organized, how changes are handled, and how risks are managed during delivery.

- Resource Planning: Identifying and allocating the people, time, budget, and tools needed to execute the project successfully.

Example: Team composition, project timeline/milestones, cost estimates, hardware/software needs. Aligns ambition with reality—ensures the solution can actually be delivered with available means.

Strategy Questions to Ask

- **Build or buy?** – Decide whether to develop a custom solution in-house or purchase/license an existing off-the-shelf product.
- **Web, mobile, or desktop?** – Choose the primary platform(s) and delivery method based on user needs, accessibility, and context of use.
- **Centralized or distributed?** – Determine if the system will run on centralized servers or across decentralized nodes/networks.
- **Cloud or on-premise?** – Decide between cloud-based hosting for scalability and flexibility, or on-premise hosting for control and security.
- **Which programming language?** – Select a language that balances project needs, team expertise, performance, and ecosystem support.
- **Which framework?** – Choose development frameworks and tools that accelerate development while ensuring maintainability and scalability.

Development Model Selection

Common models:

- **Waterfall** – A linear, sequential approach where each phase must be completed fully before the next begins, best for projects with well-defined, stable requirements.
- **Agile** – An iterative, flexible methodology emphasizing collaboration, customer feedback, and small, frequent releases, ideal for dynamic or evolving requirements.
- **Spiral** – A risk-driven iterative model that combines prototyping with staged development, focusing on risk assessment and mitigation in each cycle.
- **Iterative** – A cyclical approach where the project is built in repeated cycles (iterations), allowing refinement and addition of features gradually.
- **DevOps** – A culture and practice emphasizing continuous integration, delivery, and collaboration between development and operations teams throughout the software lifecycle.

Risk Consideration in Strategy

Every solution strategy must consider:

- **Technical Risks:** The risk that chosen technologies, architecture, or designs may fail, underperform, or be unsuitable for the problem.

Examples: Integration challenges with legacy systems, Scalability or performance limitations, Overly complex architecture

- **Budget Risks:** The risk that costs will exceed available funding, either through underestimation or uncontrolled changes.

Examples: Underestimated licensing/tooling costs, Hidden infrastructure expenses, Unplanned scope expansion, Vendor price increases

- **Time Risks:** The risk that the project will not be completed within planned timelines, delaying value delivery.

Examples: Overly optimistic estimates Scope creep without timeline adjustment. Team velocity slower than expected

Risk Consideration in Strategy

- **Resource Risks:** The risk that necessary human resources, skills, or equipment won't be available when needed.

Examples: Key team members leaving, Skill gaps in new technologies, Hardware/software procurement delays

- **Security Risks:** The risk that the solution will introduce vulnerabilities or fail to meet security requirements.

Examples: Inadequate authentication/authorization, Data exposure or privacy violations

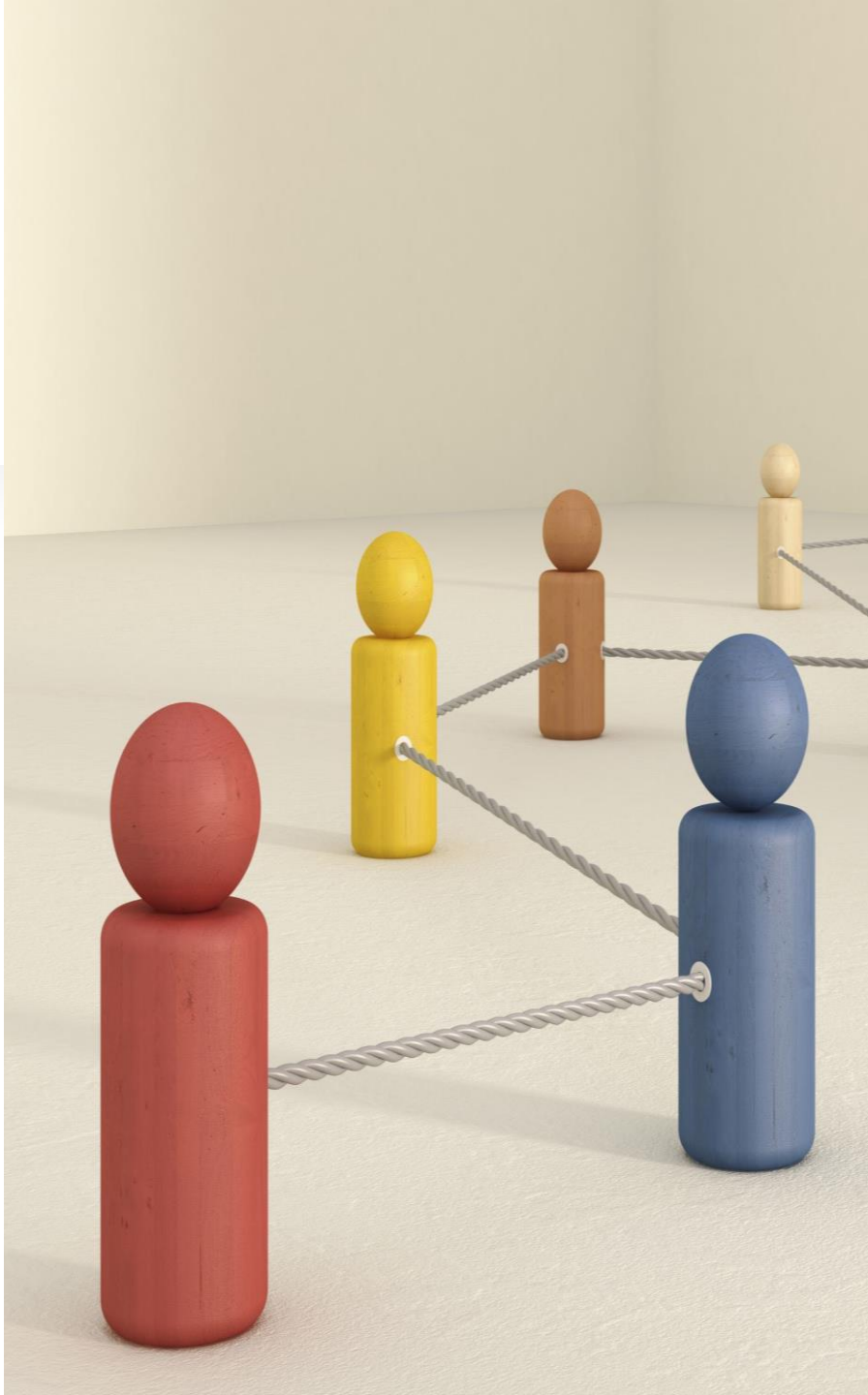
Resource Planning

Resources include:

- **Developers:** Skilled personnel, roles, and team composition needed to design, build, and test the software.
- **Tools:** Software, licenses, platforms, and hardware required for development, testing, and project management.
- **Budget:** Financial allocation covering salaries, tools, infrastructure, training, and contingency costs.
- **Time:** Project timeline, schedules, deadlines, and availability windows for both people and systems.
- **Infrastructure:** Servers, networks, cloud services, and physical or virtual environments needed for development, deployment, and hosting.

Common Planning Mistakes

- **Starting without requirements** – Beginning development or design before needs, goals, and constraints are clearly defined and agreed upon.
- **Ignoring stakeholders** – Failing to involve key users, clients, or decision-makers in the planning process, leading to misaligned expectations.
- **Underestimating complexity** – Assuming the solution is simpler than it truly is, leading to unrealistic timelines, budgets, and scope.
- **No risk analysis** – Proceeding without identifying, assessing, or preparing for potential obstacles and uncertainties.
- **No documentation** – Failing to record decisions, requirements, and plans, causing confusion, inconsistency, and knowledge loss over time.



THANK YOU.

ANY QUESTIONS?